



PROJETO E ANÁLISE DE ALGORITMOS

AULA 10 – PROGRAMAÇÃO DINÂMICA

10.0 TÉCNICAS AVANÇADAS DE PROJETO E ANÁLISE

- Vimos de projeto de algoritmos que possuem extensa aplicação, como divisão e conquista, aleatorização e solução de recorrências. As técnicas que veremos nas próximas aulas são um pouco mais sofisticadas, mas nos ajudam a abordar muitos problemas de computação.
- Veremos:
 - Programação dinâmica
 - Algoritmos gulosos
 - Análise amortizada

10.0 TÉCNICAS AVANÇADAS DE PROJETO E ANÁLISE

- A programação dinâmica se aplica a problemas de otimização nos quais fazemos um conjunto de escolhas para chegar a uma **solução ótima**, cada escolha gerando subproblemas da mesma forma que o problema original, e os mesmos subproblemas surgem repetidamente.
- A estratégia fundamental é **armazenar a solução** para cada um desses subproblemas, **em vez de recalculá-la**. Veremos como essa ideia simples, por vezes, pode **transformar algoritmos de tempo exponencial em algoritmos de tempo polinomial**.

10.0 TÉCNICAS AVANÇADAS DE PROJETO E ANÁLISE

- Como os algoritmos de programação dinâmica, os algoritmos gulosos, em geral, se aplicam a **problemas de otimização** nos quais fazemos diversas escolhas para chegar a uma solução ótima.
- A ideia de um algoritmo guloso é fazer cada escolha de maneira **ótima local**, resultando em um algoritmo **mais rápido** do que aquele obtido com a programação dinâmica.

10.0 TÉCNICAS AVANÇADAS DE PROJETO E ANÁLISE

- Usamos análise amortizada para analisar certos algoritmos que executam uma sequência de **operações semelhantes**. Em vez de estimar o custo da sequência de operações limitando o custo real de cada operação separadamente, uma análise amortizada impõe limite para o **custo real** da sequência inteira.
- Uma vantagem dessa abordagem é que, embora algumas operações possam ser caras, muitas outras podem ser mais acessíveis. Podemos usar a análise amortizada no projeto de algoritmos, já que o projeto de um algoritmo e a análise de seu tempo de execução com frequência estão intimamente relacionados.

10.1 PROGRAMAÇÃO DINÂMICA

- A programação dinâmica, assim como o método de divisão e conquista, resolve problemas combinando as soluções para subproblemas. (Nesse contexto, “programação” se refere a um método tabular, não ao processo de escrever código de computador.)
- Os algoritmos de divisão e conquista subdividem o problema em subproblemas independentes, resolvem os subproblemas recursivamente e depois combinam suas soluções para resolverem o problema original.

10.1 PROGRAMAÇÃO DINÂMICA

- Ao contrário, a programação dinâmica se aplica quando os subproblemas se sobrepõem, isto é, quando os subproblemas compartilham subsubproblemas.
- Nesse contexto, um algoritmo de divisão e conquista trabalha mais que o necessário, resolvendo repetidamente os subsubproblemas comuns. Um algoritmo de programação dinâmica resolve cada subsubproblema apenas uma vez e depois grava sua resposta em uma tabela, evitando, assim, o trabalho de recalcular a resposta toda vez que resolver cada subsubproblema.

10.1 PROGRAMAÇÃO DINÂMICA

- Em geral, aplicamos a programação dinâmica em problemas de otimização.
- Tais problemas podem ter muitas soluções possíveis. Cada solução tem um valor, e desejamos encontrar uma solução com o valor ótimo (mínimo ou máximo).
- Denominamos tal solução “uma” solução ótima para o problema, ao contrário de “a” solução ótima, já que podem existir várias soluções que alcançam o valor ótimo.

10.1 PROGRAMAÇÃO DINÂMICA

- O desenvolvimento de um algoritmo de programação dinâmica segue uma sequência de quatro etapas:
 1. Caracterizar a estrutura de uma solução ótima.
 2. Definir recursivamente o valor de uma solução ótima.
 3. Calcular o valor de uma solução ótima, normalmente de baixo para cima.
 4. Construir uma solução ótima com as informações calculadas.

10.2 CORTE DE HASTES

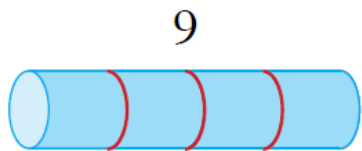
- O problema do corte de hastes consiste em decidir onde cortar hastes de aço. Uma empresa compra hastes de aço longas e as corta em hastes mais curtas, para vendê-las. Cada corte é livre.
- A direção da empresa precisa saber qual é o **melhor modo de cortar as hastes**.
- A empresa possui uma tabela indicando, para $i = 1, 2, \dots, n$, o preço p_i em dólares que a empresa cobra por uma haste de i polegadas de comprimento. O comprimento de cada haste é sempre um número inteiro de polegadas.

10.2 CORTE DE HASTES

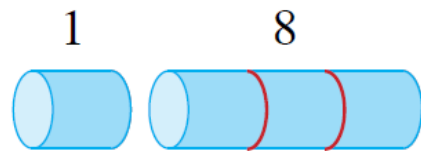
- Dada uma haste de n polegadas de comprimento e uma tabela de preços p_i para $i = 1, 2, \dots, n$, determine a receita máxima r_n que se pode obter cortando a haste e vendendo os pedaços.
- Observe que, se o preço p_n para uma haste de comprimento n for suficientemente grande, uma solução ótima pode exigir que ela não seja cortada.

10.2 CORTE DE HASTES

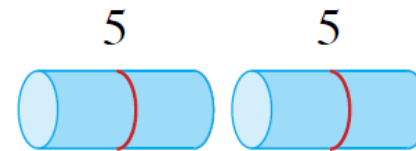
tamanho i	1	2	3	4	5	6	7	8	9	10
preço p_i	1	5	8	9	10	17	17	20	24	30



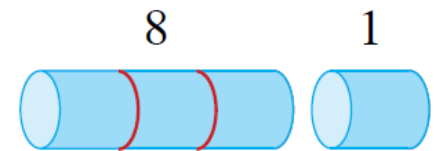
(a)



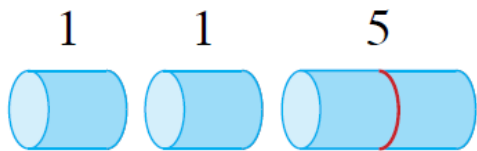
(b)



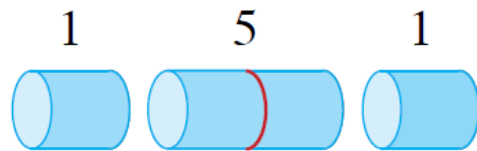
(c)



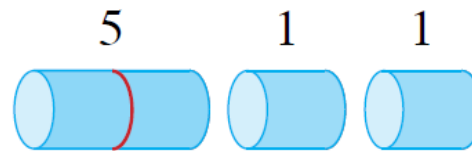
(d)



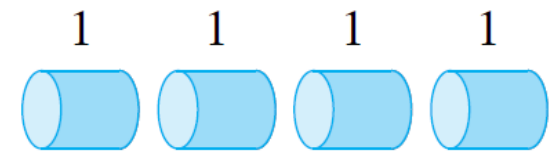
(e)



(f)



(g)



(h)

10.2 CORTE DE HASTES

- Vemos que cortar uma haste de 4 polegadas em duas peças de 2 polegadas produz a receita $p_2 + p_2 = 5 + 5 = 10$, que é a solução ótima.
- A empresa pode cortar uma haste de comprimento n de 2^{n-1} modos diferentes, já que há a opção independente de cortar ou não cortar, à distância de i polegadas da extremidade esquerda, para $i = 1, 2, \dots, n - 1$.
- Indicamos um desdobramento em pedaços usando notação de adição comum, portanto, $7 = 2 + 2 + 3$ indica que uma haste de comprimento 7 foi cortada em três peças — duas de comprimento 2 e uma de comprimento 3.

10.2 CORTE DE HASTES

- Se uma solução ótima cortar a haste em k pedaços, para algum $1 \leq k \leq n$, então o desdobramento ótimo:

$$n = i_1 + i_2 + \cdots + i_k$$

da haste em peças de comprimentos $i_1, i_2, \dots, i_k$ gera a receita máxima correspondente

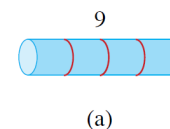
$$r_n = p_{i_1} + p_{i_2} + \cdots + p_{i_k}$$

10.2 CORTE DE HASTES

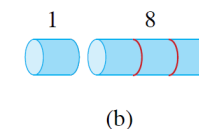
tamanho i	1	2	3	4	5	6	7	8	9	10
preço p_i	1	5	8	9	10	17	17	20	24	30

- No exemplo da figura anterior, podemos determinar os números da receita ótima r_i , para $i = 1, 2, \dots, 10$, por inspeção, com os correspondentes desdobramentos ótimos:

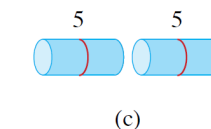
- $r_1 = 1$ pela solução $1 = 1$ (nenhum corte),



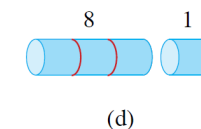
- $r_2 = 5$ pela solução $2 = 2$ (nenhum corte),



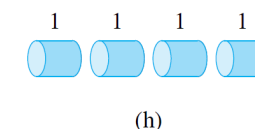
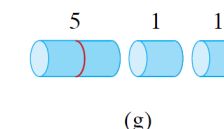
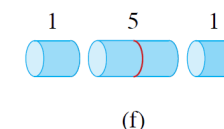
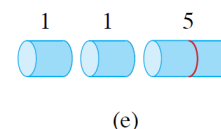
- $r_3 = 8$ pela solução $3 = 3$ (nenhum corte),



- $r_4 = 10$ pela solução $4 = 2 + 2$,



- $r_5 = 13$ pela solução $5 = 2 + 3$,

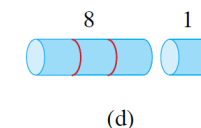
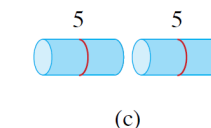
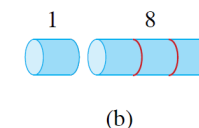
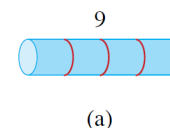


10.2 CORTE DE HASTES

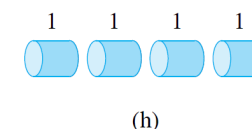
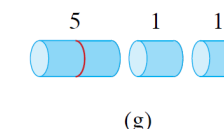
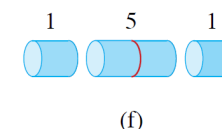
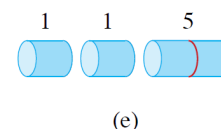
tamanho i	1	2	3	4	5	6	7	8	9	10
preço p_i	1	5	8	9	10	17	17	20	24	30

- No exemplo da figura anterior, podemos determinar os números da receita ótima r_i , para $i = 1, 2, \dots, 10$, por inspeção, com os correspondentes desdobramentos ótimos:

- $r_6 = 17$ pela solução $6 = 6$ (nenhum corte)



- $r_7 = 18$ pela solução $7 = 1 + 6$ ou $7 + 2 + 2$



- $r_8 = 22$ pela solução $8 = 2 + 6$,

- $r_9 = 25$ pela solução $9 = 3 + 6$,

- $r_{10} = 30$ pela solução $10 = 10$ (nenhum corte).

10.2 CORTE DE HASTES

- De modo mais geral, podemos enquadrar os valores r_n para $n \geq 1$ em termos de receitas ótimas advindas de hastes mais curtas:

$$r_n = \max\{pn, r_1 + r_{n-1}, r_2 + r_{n-2}, \dots, r_{n-1} + r_1\}$$

- O primeiro argumento, pn , corresponde a não fazer nenhum corte e vender a haste de comprimento n como tal.
- Os outros $n - 1$ argumentos para $\max$ correspondem à receita máxima obtida de um corte inicial da haste em duas peças de tamanhos i e $n - i$, para cada $i = 1, 2, \dots, n - 1$ e, então prosseguindo com o corte ótimo dessas peças, obtendo as receitas r_i e r_{n-i} dessas duas peças.

10.2 CORTE DE HASTES

- Como não sabemos de antemão qual é o valor de i que otimiza a receita, temos de considerar todos os valores possíveis para i e escolher aquele que maximize a receita.
- Temos também a opção de não escolher nenhum i se pudermos obter mais receita vendendo a haste sem cortes.
- Observe que, para resolvermos o problema original de tamanho n , resolvemos **problemas menores** do mesmo tipo, porém de tamanhos menores.

10.2 CORTE DE HASTES

- Uma vez executado o primeiro corte, podemos considerar os dois pedaços como **instâncias independentes** do problema do corte da haste.
- A solução ótima global incorpora soluções ótimas para os dois subproblemas relacionados, maximizando a receita gerada por esses dois pedaços.
- Dizemos que o problema do corte de hastes exhibe **subestrutura ótima**: soluções ótimas para um problema incorporam soluções ótimas para subproblemas relacionados, que podemos resolver independentemente.

10.2 CORTE DE HASTES

- Um modo relacionado, mas ligeiramente mais simples, de arranjar uma estrutura recursiva para o problema do corte de hastes considera que um desdobramento consiste em uma primeira peça de comprimento i cortada da extremidade esquerda e o que restou do lado direito, com comprimento $n - i$.
- Somente o resto, e não a primeira peça, pode **continuar a ser dividido**.
- Podemos considerar cada desdobramento de uma haste de comprimento n desse modo: uma primeira peça seguida por algum desdobramento do resto.

10.2 CORTE DE HASTES

- Quando fazemos isso, podemos expressar a solução que não contém nenhum corte dizendo que a primeira peça tem tamanho $i = n$ e receita p_n e que o resto tem tamanho 0 com receita correspondente $r_0 = 0$.
- Assim, obtemos a seguinte versão mais simples da equação:

$$r_n = \max\{pi + r_{n-i} : 1 \leq i \leq n\}$$

- Nessa formulação, uma solução ótima incorpora a solução para somente um subproblema relacionado — **o resto** — em vez de dois.

10.2 CORTE DE HASTES

- O procedimento CORTA-HASTE implementa o cálculo implícito na equação anterior de modo direto, recursivo, de cima para baixo.
- O procedimento CORTA-HASTE toma como entrada um vetor $p[1:n]$ de preços e um inteiro n , e retorna a máxima receita possível para uma haste de comprimento n .

10.2 CORTE DE HASTES

CORTA-HASTE(p, n)

1 if $n == 0$

2 return 0

3 $q = -\infty$

4 for $i = 1$ to n

5 $q = \text{máx}\{q, p[i] + \text{CORTA-HASTE}(p, n - i)\}$

6 return q

10.2 CORTE DE HASTES

- Se $n = 0$, nenhuma receita é possível e, portanto, CORTA-HASTE retorna 0 na linha 2.
- A linha 3 inicializa a receita máxima q para $-\infty$, de modo que o laço for na linhas 4–5 calcula corretamente $q = \text{máx}\{p_i + \text{CORTA-HASTE}(p, n - i) : 1 \leq i \leq n\}$.
- Então, a linha 6 retorna esse valor. Uma simples indução em n prova que essa resposta é igual à resposta desejada r_n , dada pela equação anterior.

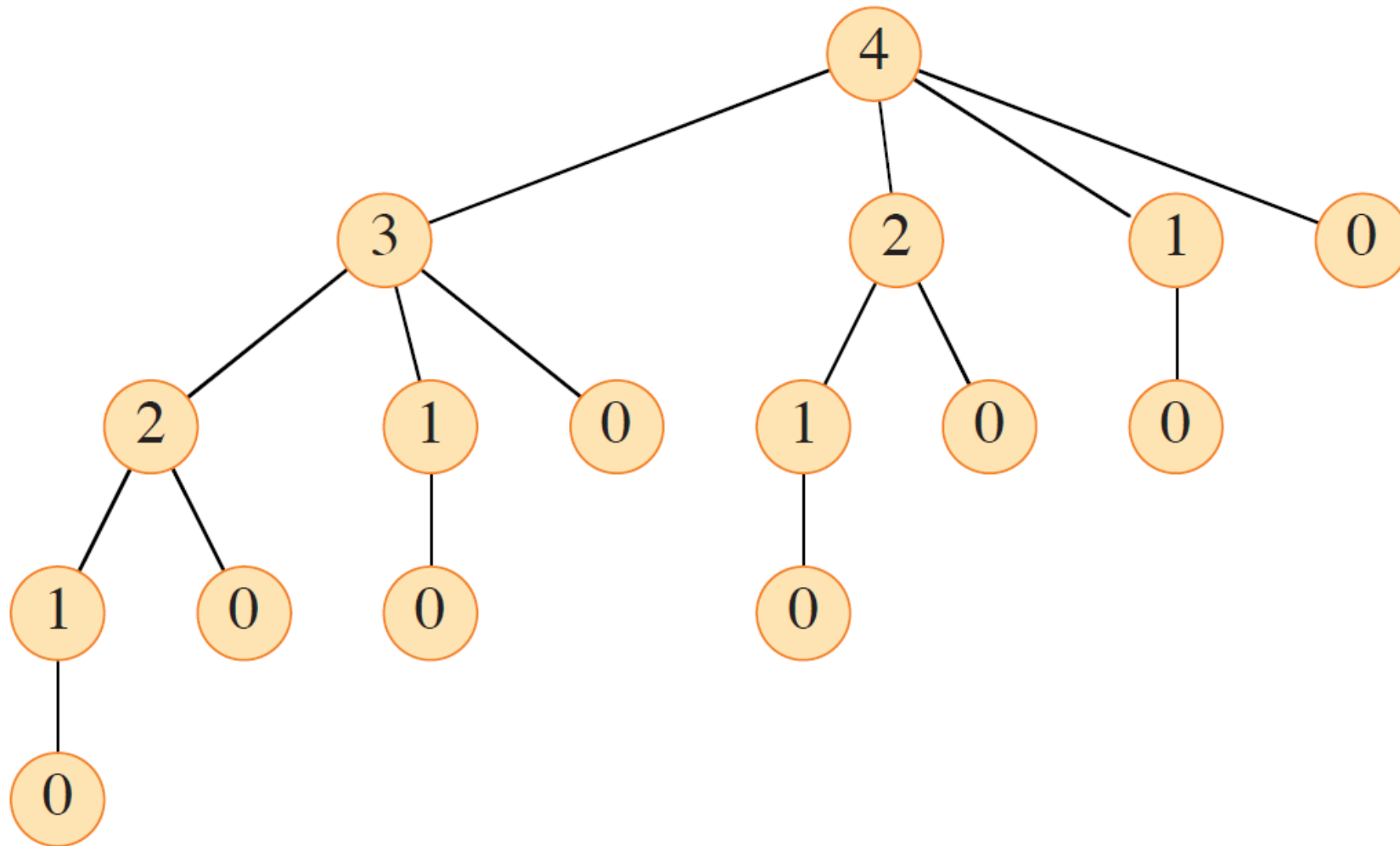
10.2 CORTE DE HASTES

- Se você tivesse de codificar CORTA-HASTE em sua linguagem de programação favorita e executá-lo em seu computador, veria que, tão logo o tamanho se torna moderadamente grande, seu programa leva um longo tempo para executá-lo.
- Para $n = 40$, seu programa pode demorar vários minutos e, o que é mais provável, mais de uma hora. Para grandes valores de n , você também constatará que, cada vez que n aumentar de 1, o tempo de execução de seu programa será aproximadamente duas vezes maior.

10.2 CORTE DE HASTES

- Por que CORTA-HASTE é tão ineficiente?
- O problema é que CORTA-HASTE chama a si mesmo **recursivamente** repetidas vezes com os mesmos valores de parâmetros, o que significa que ele resolve os mesmos problemas repetidamente.
- Quando esse processo se desenrola recursivamente, a quantidade de trabalho realizada, em função de n , cresce explosivamente.

10.2 CORTE DE HASTES



10.2 CORTE DE HASTES

- Para analisarmos o tempo de execução de CORTA-HASTE, indicamos por $T(n)$ o número total de chamadas feitas a CORTA-HASTE(p, n) para um valor particular de n .
- Essa expressão é igual ao número de nós em uma subárvore cuja raiz é identificada por n na árvore de recursão. A contagem inclui a chamada inicial à raiz.
- Assim, $T(0) = 1$ e

$$T(n) = 1 + \sum_{j=0}^{n-1} T(j)$$

10.2 CORTE DE HASTES

- O 1 inicial é para a chamada na raiz, e o termo $T(j)$ conta o número de chamadas (incluindo chamadas recursivas) resultantes da chamada $\text{CORTA-HASTE}(p, n - i)$, em que $j = n - i$.
- Essa recorrência tem como solução $T(n) = 2^n$. Ou seja, o tempo de execução de CORTA-HASTE é exponencial em n .
- Esse tempo de execução exponencial não é surpresa. CORTA-HASTE considera explicitamente todos os modos possíveis de cortar uma haste de comprimento n .

10.2 CORTE DE HASTES

- Agora será útil aplicar programação dinâmica para **melhorar essa solução**.
- Em vez de resolvermos os mesmos subproblemas repetidamente, como na solução recursiva ingênua, nós a adaptamos para **resolver cada problema somente uma vez**.
- Há realmente um **modo óbvio** de fazer isso: na primeira vez que resolvemos um subproblema, armazenamos sua solução. Se precisarmos nos referir novamente a esse problema mais tarde, bastará que o examinemos, em vez de o **recalcular**.

10.2 CORTE DE HASTES

- Armazenar soluções de subproblemas tem um custo: a memória adicional necessária para armazenar as soluções.
- A programação dinâmica serve como exemplo de **compromisso tempo-memória**.
- Economias de tempo podem ser espetaculares. Por exemplo, estamos prestes a usar a programação dinâmica para passar de um algoritmo de tempo exponencial do problema do corte de haste para um algoritmo de tempo $\Theta(n^2)$.

10.2 CORTE DE HASTES

- Uma abordagem de programação dinâmica é executada em tempo polinomial quando o número de problemas distintos envolvidos é polinomial no tamanho da entrada e podemos resolver cada subproblema em tempo polinomial.
- Normalmente, há dois modos equivalentes de implementar uma abordagem de programação dinâmica.
- A primeira abordagem é de **cima para baixo com memoização**.
- A segunda abordagem é o método **de baixo para cima**.

10.2 CORTE DE HASTES

- Na abordagem de cima para baixo, escrevemos o procedimento recursivamente de maneira natural, porém modificado para **salvar o resultado de cada subproblema** (normalmente, em um vetor ou tabela de espalhamento).
- Agora, o procedimento primeiro verifica se já resolveu antes esse subproblema. Se já resolveu, retorna o valor salvo, poupando computação adicional nesse nível; se ainda não resolveu, o procedimento calcula o valor da maneira usual, mas também o salva.
- Dizemos que o procedimento recursivo foi memoizado; **ele “lembra” quais resultados já calculou anteriormente.**

10.2 CORTE DE HASTES

- A segunda abordagem depende normalmente de alguma **noção natural do “tamanho” de um subproblema**, tal que resolver qualquer subproblema particular depende somente de resolver subproblemas “menores”.
- Resolvemos os subproblemas por ordem de tamanho, o menor primeiro, armazenando a solução de cada subproblema quando ele é resolvido inicialmente.
- Desse modo, ao resolvemos determinado subproblema, já existem **soluções salvas** para todos os menores subproblemas dos quais sua solução depende.

10.2 CORTE DE HASTES

- Resolvemos cada subproblema **somente uma vez** e, na primeira vez que o vemos, já estão resolvidos todos os seus subproblemas pré-requisitados.
- Essas duas abordagens produzem algoritmos com o mesmo tempo de execução assintótico, exceto em circunstâncias incomuns, quando a abordagem de cima para baixo não executa recursão propriamente dita para examinar todos os subproblemas possíveis. Muitas vezes, a abordagem de baixo para cima tem **fatores constantes muito melhores**, visto que tem menos sobrecarga para chamadas de procedimento.

10.2 CORTE DE HASTES

CORTA-HASTE-MEMOIZADO(p, n)

1 seja $r[0:n]$ um novo vetor //guarda “subsoluções”

2 for $i = 0$ to n

3 $r[i] = -\infty$

4 return CORTA-HASTE-MEMOIZADO-AUX(p, n, r)

10.2 CORTE DE HASTES

CORTA-HASTE-MEMOIZADO-AUX(p, n, r)

1 if $r[n] \geq 0$

2 return $r[n]$

3 if $n == 0$

4 $q = 0$

5 else $q = -\infty$

6 for $i = 1$ to n

7 $q = \text{máx}\{q, p[i] + \text{CORTA-HASTE-MEMOIZADO-AUX}(p, n - i, r)\}$

8 $r[n] = q$

9 return q

10.2 CORTE DE HASTES

- O procedimento principal CORTA-HASTE-MEMOIZADO inicializa um novo vetor auxiliar $r[0 : n]$ com o valor $-\infty$, pois os valores de receita conhecidos são sempre não negativos — uma escolha conveniente para indicar “desconhecido”.
- Então, CORTA-HASTE-MEMOIZADO chama sua rotina auxiliar, CORTA-HASTE-MEMOIZADO-AUX, que é apenas a versão memoizada de nosso procedimento de tempo exponencial, CORTA-HASTE.

10.2 CORTE DE HASTES

- Primeiro, ele consulta a linha 1 para verificar se o valor desejado já é conhecido; se for, a linha 2 retorna esse valor. Caso contrário, as linhas 3–7 calculam o valor desejado q na maneira usual, a linha 8 o salva em $r[n]$ e a linha 9 o retorna.
- A versão de baixo para cima de CORTA-HASTE-BAIXO-PARA-CIMA é ainda mais simples. No caso da abordagem da programação dinâmica de baixo para cima, CORTA-HASTE-BAIXO-PARA-CIMA usa a ordenação natural dos subproblemas: um subproblema de tamanho i é “menor” do que um subproblema de tamanho j se $i < j$. Assim, o procedimento resolve subproblemas de tamanhos $j = 0, 1, \dots, n$, naquela ordem.

10.2 CORTE DE HASTES

CORTA-HASTE-BAIXO-PARA-CIMA(p , n)

1 seja $r[0:n]$ um novo vetor

2 $r[0] = 0$

3 for $j = 1$ to n

4 $q = -\infty$

5 for $i = 1$ to j

6 $q = \text{máx}\{q, p[i] + r[j - i]\}$

7 $r[j] = q$

8 return $r[n]$

10.2 CORTE DE HASTES

- A linha 1 do procedimento CORTA-HASTE-BAIXO-PARA-CIMA gera um novo vetor $r[0 : n]$ no qual salva os resultados dos subproblemas, e a linha 2 inicializa $r[0]$ com 0, visto que uma haste de comprimento 0 não rende nenhuma receita.
- As linhas 3–6 resolvem cada subproblema de tamanho j , para $j = 1, 2, \dots, n$, em ordem crescente de tamanho.

10.2 CORTE DE HASTES

- A abordagem usada para resolver um problema de determinado tamanho j é a mesma usada por CORTA-HASTE, exceto que agora a linha 6 referencia diretamente a entrada $r[j - i]$ em vez de fazer uma chamada recursiva para resolver o subproblema de tamanho $j - i$.
- A linha 7 salva em $r[j]$ a solução do subproblema de tamanho j .
- Por fim, a linha 8 retorna $r[n]$, que é igual ao valor ótimo r_n .

10.2 CORTE DE HASTES

- As versões de baixo para cima e de cima para baixo têm o **mesmo tempo de execução assintótico**.
- O tempo de execução do procedimento CORTA-HASTE-BAIXO-PARA-CIMA é $\Theta(n^2)$, em razão de sua estrutura de laço duplamente aninhado. O número de iterações de seu laço for interno nas linhas 5–6 forma uma série aritmética.
- O tempo de execução de sua contraparte de cima para baixo, CORTA-HASTE-MEMOIZADO, também é $\Theta(n^2)$, embora esse tempo de execução possa ser um pouco mais difícil de ver.

10.2 CORTE DE HASTES

- Como chamada recursiva para resolver um subproblema já resolvido antes retorna imediatamente, CORTA-HASTE-MEMOIZADO resolve cada subproblema apenas uma vez.
- Esse procedimento resolve subproblemas para tamanhos $0, 1, \dots, n$. Para resolver um subproblema de tamanho n , o laço for das linhas 6–7 se repete n vezes. Assim, o número total de iterações desse laço for, para todas as chamadas recursivas de CORTA-HASTE-MEMOIZADO, forma uma série aritmética que dá um total de $\Theta(n^2)$ iterações, exatamente como o laço for interno de CORTA-HASTE-BAIXO-PARA-CIMA.

10.2 CORTE DE HASTES

- Frequentemente também é interessante saber qual é a configuração da solução ótima gerada pelo algoritmo de programação dinâmica.
- Os procedimentos CORTA-HASTE-MEMOIZADO e CORTA-HASTE-BAIXO-PARA-CIMA retornam o **valor** de uma solução ótima para o problema do corte de hastes, mas **não retornam a solução propriamente dita**: uma lista de tamanhos de peças.
- **Podemos estender a abordagem da programação dinâmica** para registrar não apenas o valor ótimo calculado para cada subproblema, mas também uma escolha que levou ao valor ótimo. Com essa informação, podemos imprimir imediatamente uma solução ótima.

10.2 CORTE DE HASTES

- O procedimento CORTA-HASTE-BAIXO-PARA-CIMA-EXT calcula, para cada tamanho de haste j , não somente a receita máxima r_j , mas também s_j , o tamanho ótimo da primeira peça a ser cortada.
- Esse procedimento é semelhante a CORTA-HASTE-BAIXO-PARA-CIMA, exceto pela criação do vetor s na linha 1 e pela atualização de $s[j]$ na linha 8 para guardar o tamanho ótimo i da primeira peça a cortar ao resolver um subproblema de tamanho j .

10.2 CORTE DE HASTES

```
CORTA-HASTE-BAIXO-PARA-CIMA-EXT(p, n)
1  sejam r[0:n] e s[1:n] novos vetores
2  r[0] = 0
3  for j = 1 to n
4      q =  $-\infty$ 
5      for i = 1 to j
6          if q < p[i] + r[j - i]
7              q = p[i] + r[j - i]
8              s[j] = i
9  r[j] = q
10 return r e s
```

10.2 CORTE DE HASTES

- O procedimento IMPRIME-SOLUÇÃO-CORTA-HASTE toma como entrada um vetor $p[1 : n]$ de preços e um tamanho de haste n .
- Ele chama CORTA-HASTE-BAIXO-PARA-CIMA-EXT para calcular o vetor $s[1 : n]$ de tamanhos ótimos da primeira peça e depois imprime a lista completa de tamanho de peças conforme um desdobramento ótimo de uma haste de comprimento n .
- Uma chamada a IMPRIME-SOLUÇÃO-CORTA-HASTE(p , 10) imprimiria apenas 10, mas uma chamada com $n = 7$ imprimiria os cortes 1 e 6, correspondentes ao primeiro desdobramento ótimo para r_7 dado anteriormente.

10.2 CORTE DE HASTES

IMPRIME-SOLUÇÃO-CORTA-HASTE(p, n)

1 (r, s) = CORTA-HASTE-BAIXO-PARA-CIMA-EXT(p, n)

2 while $n > 0$

3 imprime $s[n]$

4 $n = n - s[n]$

10.2 CORTE DE HASTES

tamanho i	1	2	3	4	5	6	7	8	9	10
preço p_i	1	5	8	9	10	17	17	20	24	30

- Para a tabela de preços acima, a chamada CORTA-HASTE-BAIXO-PARA-CIMA-EXT(p , 10) retorna os seguintes vetores:

i	0	1	2	3	4	5	6	7	8	9	10
$r[i]$	0	1	5	8	10	13	17	18	22	25	30
$s[i]$	0	1	2	3	2	2	6	1	2	3	10

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- Nosso próximo exemplo de programação dinâmica é um algoritmo que resolve o problema de multiplicação de cadeias de matrizes.
- Temos uma sequência (cadeia) $\langle A_1, A_2, \dots, A_n \rangle$ de n matrizes para multiplicar e desejamos calcular o produto $A_1 A_2 \cdots A_n$ usando o algoritmo-padrão para multiplicação de matrizes retangulares, enquanto **minimizamos o número de multiplicações escalares**.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- Podemos avaliar a expressão usando o algoritmo-padrão para multiplicação de pares de matrizes como uma sub-rotina, tão logo a tenhamos parentizado para resolver todas as ambiguidades relativas à multiplicação das matrizes entre si.
- A multiplicação de matrizes é **associativa** e, portanto, o produto entre elas é sempre o mesmo.
- Um produto de matrizes é **totalmente parentizado** se for uma única matriz ou o produto de dois produtos de matrizes totalmente parentizadas também expresso entre parênteses.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- Por exemplo, se a cadeia de matrizes é $\langle A_1, A_2, A_3, A_4 \rangle$, podemos expressar o produto $A_1A_2A_3A_4$ como totalmente parentizado de cinco modos distintos:
 - $(A_1(A_2(A_3A_4)))$
 - $(A_1((A_2A_3)A_4))$
 - $((A_1A_2)(A_3A_4))$
 - $((A_1(A_2A_3))A_4)$
 - $((A_1A_2)A_3)A_4$

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- O **modo** como colocamos parênteses em uma cadeia de matrizes pode ter impacto expressivo sobre o custo de avaliação do produto. Considere primeiro o custo de multiplicar duas matrizes.
- O algoritmo-padrão é dado pelo procedimento MULTIPLICA-MATRIZ-RETANGULAR que calcula $C = C + A \cdot B$ para três matrizes $A = (a_{ij})$, $B = (b_{ij})$ e $C = (c_{ij})$, em que A é $\mathbf{p} \times \mathbf{q}$, B é $\mathbf{q} \times \mathbf{r}$ e C é $\mathbf{p} \times \mathbf{r}$.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

MULTIPLICA-MATRIZ-RETANGULAR(A, B, C, p, q, r)

1 for i = 1 to p

2 for j = 1 to r

3 for k = 1 to q

4 c_{ij} = c_{ij} + a_{ik} · b_{kj}

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- O tempo de execução de MULTIPLICA-MATRIZ-RETANGULAR é dominado pelo número de multiplicações escalares na linha 4, que é pqr .
- Portanto, vamos considerar o custo da multiplicação de matrizes como o número de multiplicações escalares.
- Para ilustrar os diferentes custos incorridos pelas diferentes posições dos parênteses em um produto de matrizes, considere o problema de uma cadeia $\langle A_1, A_2, A_3 \rangle$ de três matrizes. Suponha que as dimensões das matrizes sejam 10×100 , 100×5 e 5×50 , respectivamente.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- Se multiplicarmos as matrizes de acordo com a posição dos parênteses $((A_1A_2)A_3)$, executaremos $10 \cdot 100 \cdot 5 = 5.000$ multiplicações escalares para calcular o produto 10×5 de matrizes A_1A_2 , mais outras $10 \cdot 5 \cdot 50 = 2.500$ multiplicações escalares para multiplicar essa matriz por A_3 , produzindo um total de 7.500 multiplicações escalares.
- A multiplicação de acordo com a posição alternativa dos parênteses $(A_1(A_2 A_3))$ executa $100 \cdot 5 \cdot 50 = 25.000$ multiplicações escalares para calcular o produto 100×50 de matrizes A_2A_3 , mais outras $10 \cdot 100 \cdot 50 = 50.000$ multiplicações escalares para multiplicar A_1 por essa matriz, resultando em um total de 75.000 multiplicações escalares. Assim, o cálculo do produto de acordo com a primeira posição dos parênteses é 10 vezes mais rápido.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- Enunciamos o problema de multiplicação de cadeias de matrizes da seguinte maneira: dada uma cadeia $\langle A_1, A_2, \dots, A_n \rangle$ de n matrizes em que, para $i = 1, 2, \dots, n$, a matriz A_i tem dimensão $p_{i-1} \times p_i$, expresse o produto $A_1, A_2, \dots, A_n$ como um produto totalmente parentizado de modo a minimizar o número de multiplicações escalares. A entrada é a sequência de dimensões $\langle p_0, p_1, p_2, \dots, p_n \rangle$.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- No problema de multiplicação de cadeias de matrizes, não estamos realmente multiplicando matrizes. Nossa meta é apenas determinar uma ordem para multiplicar matrizes que tenha o custo mais baixo.
- Em geral, o tempo investido na determinação dessa ordem ótima é mais que compensado pelo tempo economizado mais tarde, quando as multiplicações de matrizes são de fato executadas (por exemplo, executar apenas 7.500 multiplicações escalares em vez de 75.000).

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- A verificação exaustiva de todas as possíveis parentizações não resulta em um algoritmo eficiente.
- Vamos representar por $P(n)$ o número de parentizações alternativas de uma sequência de n matrizes. Quando $n = 1$, a sequência consiste em apenas uma matriz e, portanto, somente um modo de parentizar totalmente o produto de matrizes.
- Quando $n \geq 2$, um produto de matrizes totalmente parentizado é o produto de dois subprodutos de matrizes totalmente parentizados, e a separação entre os dois subprodutos pode ocorrer entre a k -ésima e a $(k + 1)$ -ésima matrizes para qualquer $k = 1, 2, \dots, n - 1$.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- Assim, obtemos a recorrência:

$$P(n) = \begin{cases} 1 & \text{se } n = 1 \\ \sum_{k=1}^{n-1} P(k)P(n-k) & \text{se } n \geq 2 \end{cases}$$

- A solução dessa recorrência é $\Omega(2^n)$. Portanto, o número de soluções é exponencial em n e o método de força bruta de busca exaustiva é uma estratégia ruim para determinar a parentização ótima de uma cadeia de matrizes.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- Usaremos o método da programação dinâmica para determinar a parentização ótima de uma cadeia de matrizes, acompanhando a sequência de quatro etapas declaradas no início deste capítulo:
 1. Caracterizar a estrutura de uma solução ótima.
 2. Definir recursivamente o valor de uma solução ótima.
 3. Calcular o valor de uma solução ótima.
 4. Construir uma solução ótima com as informações calculadas.
- Percorreremos essas etapas em ordem, demonstrando claramente como aplicar cada etapa ao problema.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- **Etapa 1: Estrutura de uma parentização ótima**
 - Em nossa primeira etapa do paradigma de programação dinâmica, determinamos a subestrutura ótima e depois a usamos para construir uma solução ótima para o problema partindo de soluções ótimas para subproblemas.
 - No problema de multiplicação de cadeias de matrizes, é conveniente introduzirmos primeiro a notação utilizada.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- Seja $A_{i:j}$, em que $i \leq j$, indica a matriz que resulta da avaliação do produto $A_i A_{i+1} \dots A_j$. Observe que, se o problema é não trivial, isto é, se $i < j$, então, para parentizar o produto $A_i A_{i+1} \dots A_j$ temos de separá-lo entre A_k e A_{k+1} para algum inteiro k no intervalo $i \leq k < j$.
- Isto é, para algum valor de k , primeiro calculamos as matrizes $A_{i:k}$ e $A_{k+1:j}$, e depois multiplicamos as duas para gerar o produto final $A_{i:j}$. O custo dessa parentização é o custo de calcular a matriz $A_{i:k}$, mais o custo de calcular $A_{k+1:j}$, mais o custo de multiplicá-las uma pela outra.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- A subestrutura ótima: suponha que para efetuar a parentização ótima de $A_i A_{i+1} \dots A_j$ separamos o produto entre A_k e A_{k+1} . Então, o modo como posicionamos os parênteses na subcadeia “prefixo” $A_i A_{i+1} \dots A_k$ dentro dessa parentização ótima de $A_i A_{i+1} \dots A_j$ deve ser uma parentização ótima de $A_i A_{i+1} \dots A_k$.
- Se existisse um modo menos dispendioso de parentizar $A_i A_{i+1} \dots A_k$, então poderíamos substituir essa parentização na parentização ótima de $A_i A_{i+1} \dots A_j$ para produzir outro modo de parentizar $A_i A_{i+1} \dots A_j$ cujo custo seria mais baixo que o custo ótimo: uma contradição.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- Observação semelhante é válida para parentizar a subcadeia $A_{k+1}A_{k+2}\dots A_j$ na parentização ótima de $A_iA_{i+1}\dots A_j$: ela deve ser uma parentização ótima de $A_{k+1}A_{k+2}\dots A_j$.
- Podemos construir uma solução ótima para uma instância do problema de multiplicação de cadeias de matrizes **separando o problema em dois subproblemas** (pela parentização ótima de $A_iA_{i+1}\dots A_k$ e $A_{k+1}A_{k+2}\dots A_j$), determinando soluções ótimas para instâncias de subproblemas e depois combinando essas soluções ótimas de subproblemas.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

■ Etapa 2: Uma solução recursiva

- Em seguida, definimos recursivamente o custo de uma solução ótima em termos das soluções ótimas para subproblemas.
- No caso do problema de multiplicação de cadeias de matrizes, escolhemos como nossos subproblemas os problemas da determinação do custo mínimo da parentização de $A_i A_{i+1} \dots A_j$ para $1 \leq i \leq j \leq n$.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- Dadas as dimensões de entrada $\langle p_0, p_1, p_2, \dots, p_n \rangle$, um par de índices i, j especifica um subproblema.
- Seja $m[i, j]$ o número mínimo de multiplicações escalares necessárias para calcular a matriz $A_{i:j}$. Para o problema completo, o custo mínimo para calcular $A_{1:n}$ seria, portanto, $m[1, n]$.
- Se $i = j$, o problema é trivial; a cadeia consiste em apenas uma matriz $A_{i:i} = A_i$, de modo que nenhuma multiplicação escalar é necessária para calcular o produto.
- Assim, $m[i, i] = 0$ para $i = 1, 2, \dots, n$.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- Para calcularmos $m[i, j]$ quando $i < j$, tiramos proveito da estrutura de uma solução ótima da etapa 1. Vamos considerar que, para obtermos a parentização ótima, separamos o produto $A_i A_{i+1} \dots A_j$ entre A_k e A_{k+1} , em que $i \leq k < j$.
- Então, $m[i, j]$ é igual ao custo mínimo $m[i, k]$ para calcular os subprodutos $A_{i:k}$, mais o custo mínimo $m[k + 1, j]$ para calcular o subproduto, $A_{k+1:j}$, mais o custo de multiplicar essas duas matrizes.
- Recordando que cada matriz A_i é $p_{i-1} \times p_i$, vemos que o cálculo do produto de matrizes $A_{i:k} A_{k+1:j}$ exige $p_{i-1} p_k p_j$ multiplicações escalares.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- Assim, obtemos:

$$m[i, j] = m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j$$

- Essa equação recursiva supõe que conhecemos o valor de k , o que não é verdade — não ainda. Você precisa experimentar todos os valores possíveis de k .
- Quantos deles existem? Somente $j - i$, a saber, $k = i, i + 1, \dots, j - 1$. Visto que a parentização ótima deve usar um desses valores para k , precisamos apenas verificar todos eles para determinarmos o melhor.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- Assim, nossa definição recursiva para o custo mínimo de colocar o produto $A_i A_{i+1} \dots A_j$ entre parênteses se torna:

$$m[i, j] = \begin{cases} 0 & \text{se } i = j \\ \min\{m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j : i \leq k < j\} & \text{se } i < j \end{cases}$$

- Os valores $m[i, j]$ dão os custos de soluções ótimas para subproblemas, mas não dão todas as informações de que necessitamos para construir uma solução ótima.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- Para conseguirmos fazer isso, definimos $s[i, j]$ como um valor de k no qual separamos o produto $A_i A_{i+1} \dots A_j$ para obtermos uma parentização ótima. Isto é, $s[i, j]$ é igual a um valor k tal que
$$m[i, j] = m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j.$$

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- **Etapa 3: Cálculo dos custos ótimos**
 - Um algoritmo recursivo pode encontrar cada subproblema, muitas vezes, em diferentes ramos de sua árvore de recursão. Essa propriedade de sobrepor subproblemas é a **segunda indicação** da aplicabilidade da programação dinâmica (a subestrutura ótima é a primeira).
 - Em vez de calcularmos a solução para a recorrência recursivamente, calculamos o custo ótimo usando uma **abordagem tabular**, de baixo para cima, como no procedimento ORDENA-CADEIA-DE-MATRIZES.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- Sua entrada é uma sequência $p = \langle p_0, p_1, \dots, p_n \rangle$ de dimensões de matriz, juntamente com n , de modo que, para $i = 1, 2, \dots, n$, a matriz A_i tem dimensões $p_{i-1} \times p_i$.
- O procedimento utiliza uma tabela auxiliar $m[1:n, 1:n]$ para armazenar os custos $m[i, j]$ e outra tabela auxiliar $s[1:n - 1, 2:n]$ que registra qual índice de k alcançou o custo ótimo no cálculo de $m[i, j]$.
- Usaremos a tabela s para ajudar na construção de uma solução ótima.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

```
ORDENA-CADEIA-DE-MATRIZES(p, n)
1  sejam m[1:n, 1:n] e s[1:n - 1, 2:n] novas tabelas
2  for i = 1 to n
3      m[i, i] = 0
4  for c = 2 to n
5      for i = 1 to n - c + 1
6          j = i + c - 1
7          m[i, j] = ∞
8          for k = i to j - 1
9              q = m[i, k] + m[k + 1, j] + pi-1pkpj
10             if q < m[i, j]
11                 m[i, j] = q
12                 s[i, j] = k
13 return m e s
```

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- Sua entrada é uma sequência $p = \langle p_0, p_1, \dots, p_n \rangle$ de dimensões de matriz, juntamente com n , de modo que, para $i = 1, 2, \dots, n$, a matriz A_i tem dimensões $p_{i-1} \times p_i$.
- O procedimento utiliza uma tabela auxiliar $m[1:n, 1:n]$ para armazenar os custos $m[i, j]$ e outra tabela auxiliar $s[1:n - 1, 2:n]$ que registra qual índice de k alcançou o custo ótimo no cálculo de $m[i, j]$.
- Usaremos a tabela s para ajudar na construção de uma solução ótima.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- A recorrência mostra que, para calcular o custo do produto matricial $A_{i:j}$, primeiro é preciso calcular os custos dos produtos $A_{i:k}$ e $A_{k+1:j}$ para todo $k = i, i + 1, \dots, j - 1$.
- A cadeia $A_i A_{i+1} \dots A_j$ consiste em $j - i + 1$ matrizes, e as cadeias $A_i A_{i+1} \dots A_k$ e $A_{k+1} A_{k+2} \dots A_j$ consistem em $k - i + 1$ e $j - k$ matrizes, respectivamente. Visto que $k < j$, uma cadeia de $k - i + 1$ matrizes consiste em menos de $j - i + 1$ matrizes.
- De modo semelhante, como $k \geq i$, uma cadeia de $j - k$ matrizes consiste em menos de $j - i + 1$ matrizes.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- Vejamos como o procedimento ORDENA-CADEIA-DE-MATRIZES preenche as entradas $m[i, j]$ em ordem crescente de comprimento da cadeia.
- As linhas 2–3 inicializam $m[i, i] = 0$ para $i = 1, 2, \dots, n$, pois qualquer cadeia de matriz com apenas uma matriz não requer multiplicações escalares.
- No laço for das linhas 4–12, a variável de laço c indica o comprimento das cadeias de matrizes cujos custos mínimos estão sendo calculados. Cada iteração desse laço utiliza a recorrência no cálculo de $m[i, i + c - 1]$ para $i = 1, 2, \dots, n - c + 1$.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- Na primeira iteração, $c = 2$, e portanto, o laço calcula $m[i, i + 1]$ para $i = 1, 2, \dots, n - 1$ (os custos mínimos para cadeias de comprimento $c = 2$).
- Na segunda passagem pelo laço, o algoritmo calcula $m[i, i + 2]$ para $i = 1, 2, \dots, n - 2$ (os custos mínimos para cadeias de comprimento $c = 3$).
- E assim por diante, terminando com uma única cadeia de matrizes de comprimento $c = n$ e calculando $m[1, n]$. Quando as linhas 7–12 calculam um custo $m[i, j]$, esse custo depende apenas das entradas de tabela $m[i, k]$ e $m[k + 1, j]$, que já foram calculadas.

Matriz	A_1	A_2	A_3	A_4	A_5	A_6
Dimensão	30x35	35x15	15x5	5x10	10x20	20x25

p_0

m

p_1

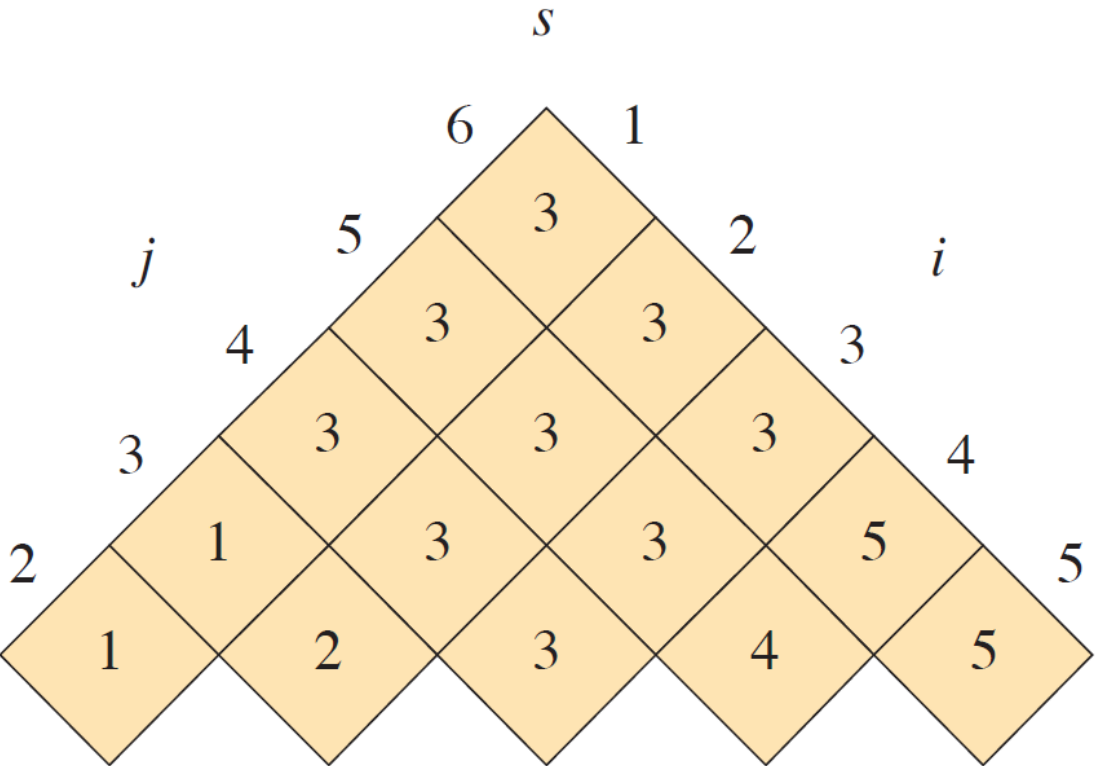
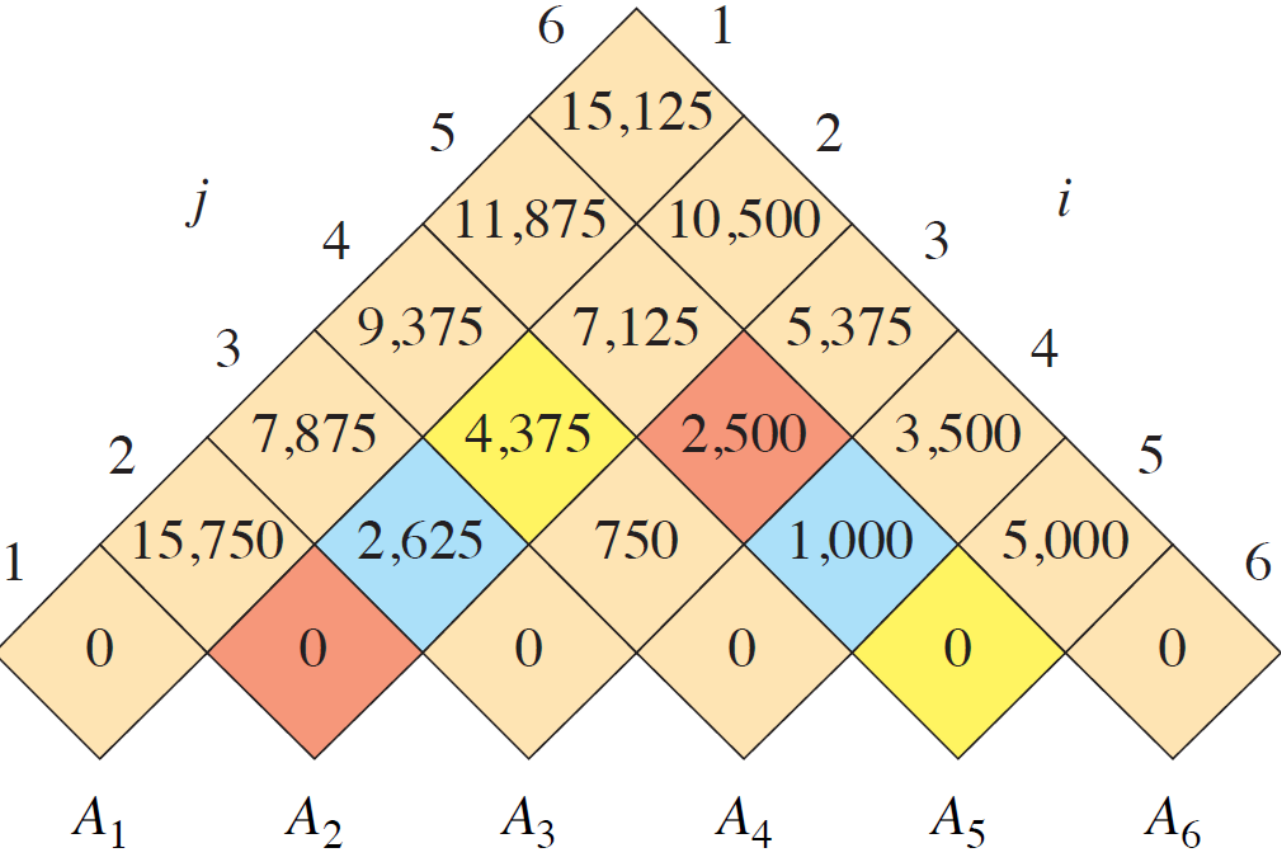
p_2

p_3

p_4

p_5

p_6



10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- A tabela mostrada na figura sofreu uma rotação para colocar a diagonal principal na posição horizontal.
- Usando essa configuração, podemos determinar o custo mínimo $m[i, j]$ para multiplicar uma subcadeia de matrizes $A_i A_{i+1} \dots A_j$ na interseção de linhas que partem de A_i na direção nordeste e de A_j na direção noroeste.
- Cada linha diagonal na tabela contém as entradas para cadeias de matrizes do mesmo comprimento.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- ORDENA-CADEIA-DE-MATRIZES calcula as linhas de baixo para cima e da esquerda para a direita dentro de cada linha.
- Calcula cada entrada $m[i, j]$ usando os produtos $p_{i-1} p_k p_j$ para $k = i, i + 1, \dots, j - 1$ e todas as entradas a sudoeste e a sudeste de $m[i, j]$.
- Uma simples inspeção da estrutura de laços encaixados de ORDENA-CADEIA-DE-MATRIZES produz um tempo de execução de $O(n^3)$ para o algoritmo. O algoritmo requer espaço $\Theta(n^2)$ para armazenar as tabelas m e s . Assim, ORDENA-CADEIA-DE-MATRIZES é muito mais eficiente que o método de tempo exponencial que enumera todas as possíveis parentizações e verifica cada uma delas.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- As tabelas sofreram rotação para colocar a diagonal principal na posição horizontal. A tabela m usa somente a diagonal principal e o triângulo superior, e a tabela s usa somente o triângulo superior. O número mínimo de multiplicações escalares para multiplicar as seis matrizes é $m[1, 6] = 15125$.
- Entre as entradas destacadas em vermelho, amarelo e azul, os pares que têm o mesmo destaque são tomados juntos na linha 9 quando se calcula $m[2,5] = 7125$.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

$$m[2, 5] = \min \begin{cases} m[2, 2] + m[3, 5] + p_1 p_2 p_5 = 0 + 2500 + 35 \cdot 15 \cdot 20 = 13,000, \\ m[2, 3] + m[4, 5] + p_1 p_3 p_5 = 2625 + 1000 + 35 \cdot 5 \cdot 20 = 7125, \\ m[2, 4] + m[5, 5] + p_1 p_4 p_5 = 4375 + 0 + 35 \cdot 10 \cdot 20 = 11,375 \end{cases}$$

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- **Etapa 4: Construção de uma solução ótima**
 - Embora determine o número ótimo de multiplicações escalares necessárias para calcular um produto de cadeias de matrizes, ORDENA-CADEIA-DE-MATRIZES não mostra diretamente como multiplicar as matrizes.
 - A tabela $s[1 : n - 1, 2 : n]$ nos dá a informação que precisamos para fazer isso.
 - Cada entrada $s[i, j]$ registra um valor de k tal que uma parentização ótima de $A_i A_{i+1} \dots A_j$ separa o produto entre A_k e A_{k+1} .

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- Assim, sabemos que a multiplicação de matrizes final no cálculo ótimo de $A_{1:n}$ é $A_{1:s[1,n]} A_{s[1,n]+1:n}$.
- A tabela s também contém as informações necessárias para determinar as multiplicações de matrizes anteriores recursivamente, já que $s[1, s[1, n]]$ determina a última multiplicação de matrizes no cálculo de $A_{1:s[1,n]}$ e $s[s[1, n] + 1, n]$ determina a última multiplicação de matrizes no cálculo de $A_{s[1,n]+1:n}$.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

- O procedimento recursivo IMPRIME-PARÊNTESES-ÓTIMOS a seguir imprime uma parentização ótima do produto de cadeias de matrizes $A_i, A_{i+1}, \dots, A_j$, dada a tabela s calculada por ORDENA-CADEIA-DE-MATRIZES e os índices i e j .
- A chamada inicial IMPRIME-PARÊNTESES-Ótimos($s, 1, n$) imprime uma parentização ótima do produto de cadeias de matrizes $A_1, A_2, \dots, A_n$.
- No exemplo anterior, a chamada IMPRIME-PARÊNTESES-ÓTIMOS($s, 1, 6$) imprime a parentização ótima de $((A_1 (A_2 A_3))((A_4 A_5) A_6))$.

10.3 MULTIPLICAÇÃO DE CADEIAS DE MATRIZES

```
IMPRIME-PARÊNTESES-ÓTIMOS( $s$ ,  $i$ ,  $j$ )
```

```
1 if  $i == j$ 
```

```
2     imprime "A" $i$ 
```

```
3 else imprime "("
```

```
4     IMPRIME-PARÊNTESES-ÓTIMOS( $s$ ,  $i$ ,  $s[i, j]$ )
```

```
5     IMPRIME-PARÊNTESES-ÓTIMOS( $s$ ,  $s[i, j] + 1$ ,  $j$ )
```

```
6     imprime ")"
```

10.4 EXERCÍCIO

- **Exercício:** ler as seções 14.3 e **14.4** do Livro Algoritmos Thomas H. Cormen.